Linux Filesystem

1. What is FileSystem?

A file system is a method used by the operating system to organize, store, and manage data on a storage device like a hard disk. It defines how data is stored, accessed, and retrieved

2. Windows vs Linux

Windows path: C:\Users\<user_name>\Downloads

Linux: /home/<user_name>/<file_name>

Windows is case insensitive and Linux is case sensitive

3. Types of File Systems (Common)

ext4 → most common in Linux
xfs → used in enterprise systems
ntfs → Windows
fat32 → USB drives

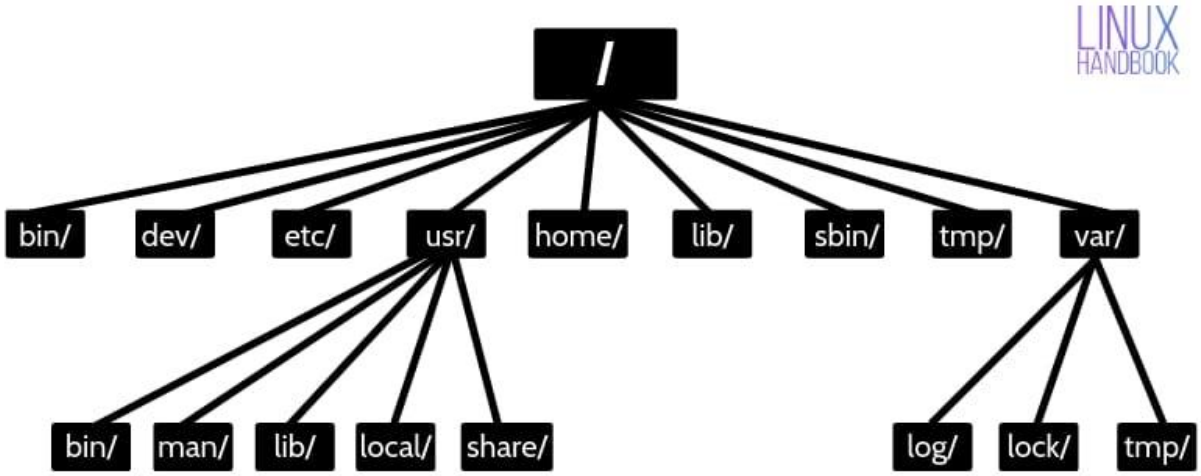
4. Difference between ext4 and XFS

ext4 and XFS are both Linux file systems. ext4 is more general-purpose and widely used, while XFS is designed for high performance and large-scale systems, especially for handling large files and parallel operations.

Key Differences

Feature	ext4	xfs
Type	General purpose	High-performance
Performance	Good	Better for large data
File Size Support	Up to ~16TB	Very large (petabytes)
Resize	Can shrink & grow	Only grow (cannot shrink)
Speed	Moderate	Faster for large files
Journaling	Yes	Yes (more efficient)
Use Case	Default Linux FS	Enterprise / Big data

5. Linux File System Structure



/

Root directory — the top of the filesystem; everything starts from here.

/bin

Essential user commands (like ls, cp, mv) required for basic system operation.

/sbin

System/admin commands used for system maintenance (mainly for root user).

/etc

Stores system-wide configuration files (users, services, network configs).

/home

Contains personal directories for all regular users.

/root

Home directory of the root (admin) user.

/var

Stores variable data like logs, cache, mail, and application data.

/tmp

Temporary files; usually cleared automatically after reboot.

/usr

Contains user-installed applications, libraries, and documentation.

/opt

Used for installing third-party or optional software.

/dev

Contains device files representing hardware (disks, USB, etc.).

/proc

Virtual filesystem showing process and system information (CPU, memory, running processes).

/boot

Contains bootloader files and Linux kernel required for system startup.

/mnt

Temporary mount point for manually mounted filesystems.

/media

Auto-mounted removable media like USB drives and CDs.

/lib & /lib64

Shared libraries required by system programs and binaries.

Users & Groups

1. What is a User?

A **user** is a person or system account that can log into Linux and perform actions.

Example:

- Root -> admin user
- Steve -> normal user

2. What is a Group?

A **group** is a collection of users.

Uses:

- To manage permission easily.
- Instead of giving access to each user, you give access to a group.

Example:

- Group: Developers
- Users: ram, shyam , alex

Commands:

```
useradd john      # create user
groupadd dev      # create group
userdel john      #delete user
groupdel dev      #delete group
usermod -aG dev john   # add user to group
gpasswd –d john dev  #remove john from dev group
```

id <user_name>

output: uid=1001(user_name) gid=1004(group_name) groups=1004(group_name),1001(dev)

User ID | Group ID

File Permissions

File Permissions (rwx)

Every file/directory has permissions that define: Who can read, write, or execute it

Three Types of Permissions:

Symbol	Meaning	Example
r	Read	View file
w	Write	Edit file
x	Execute	Run file

Three Levels of Access:

Level	Meaning
User (u)	Owner of file
Group (g)	Group members
Others (o)	Everyone else

Each permission has a number:

Permission	Value
r (read)	4
w (write)	2
x (execute)	1

Example1:

-rwxr-xr--

Section	**Meaning**
Rwx	Owner → full access
r-x	Group → read & execute
r--	Others → read only

Easy Trick: Always read in groups of 3:

rwx | r-x | r--

u g o

<u>Change Persmission:</u>

chmod 755 file.txt

7 → rwx

5 → r-x

5 → r-x

<u>Example2:</u>

mkdir /tmp/shared

chmod 1777 /tmp/shared

1 777

| └── Normal permissions (rwx for all)

└─── Special permission (sticky bit)

Owner → full access

Group → full access

Others → full access

So everyone can read, write, create files

What is that 1? (Sticky Bit)

1 = Sticky Bit

What Sticky Bit does

Rule:

Users can create files, but can only delete their own files

<u>Example3:</u>

chmod g+s /tmp/project

This command sets the SGID (Set Group ID) on the /tmp/project directory.

What SGID does?

When SGID is applied on a directory:

All new files and folders created inside will inherit the same group as the directory.

Without SGID

/project → group = dev

User john creates file:

file.txt → group = john

Problem:

Group mismatch

Team cannot access properly

With SGID (After chmod g+s)

/project → group = dev

User john creates file:

file.txt → group = dev

Solution:

All team members (dev group) can access

Ownership

Every file has:

- Owner (user)
- Group

Example1:

-rw-r--r-- 1 john dev file.txt

- Owner = john
- Group = dev

Change Ownership

- chown john file.txt **# change owner**
- chown john:dev file.txt **# change owner + group**

Example2:

-rwxr----- 1 john dev script.sh

- Owner (john) → full access
- Group (dev) → read only
- Others → no access

Example3:

- chown john:dev file.txt
- chmod 770 file.txt
- Owner + group → full access
- Others → no access

Users = who is using system

Groups = collection of users

Permissions = what they can do

Ownership = who controls the file